SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:		**Reg. No.:**	
Section / Batch:		**Date:**	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
	GRAND TOTAL:	**100 Marks**	

SECTION A – APPLICATION BASED QUESTIONS

Q1.

Q2.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ASSIGNMENT ANSWER

Name: VIGNESH Y | Reg. No.: 192524100

A delivery company maintains GPS coordinates of delivery points and computes distances between every pair to find the closest drop location. As the number of locations increases, delays are observed.

a) Brute-Force Closest Pair Algorithm — How It Operates

In this delivery scenario, each delivery point is represented as a coordinate pair (x, y) obtained from GPS data. The brute-force approach works as follows:

1. Store all points: Collect all n delivery locations as (x, y) coordinates in a list or array.
2. Compare every pair: Use two nested loops — the outer loop picks a point i, and the inner loop compares it against every other point j (where j ≠ i).
3. Compute distance: For each pair (i, j), calculate the Euclidean distance using the distance formula shown below.

$$d(i, j) = \sqrt{[\ (x_i - x_j)^2 + (y_i - y_j)^2\ ]}$$

4. Track the minimum: Maintain a running variable storing the smallest distance found so far. If a new pair's distance is smaller, update it.
5. Return the result: After all pairs are checked, the algorithm returns the pair with the minimum distance as the "closest pair."

Essentially, the algorithm exhaustively checks every possible pair of delivery points without skipping any, guaranteeing correctness but at a high computational cost.

b) Time Complexity Derivation

For n delivery points, the number of unique pairs to compare is given by the combination formula:

$$C(n, 2) = n(n - 1) / 2$$

Since each distance calculation (subtraction, squaring, addition, square root) takes constant time O(1), the total time is:

$$T(n) = [\ n(n - 1) / 2\ ] \times O(1) = O(n^2)$$

Derivation via nested loops:

- Outer loop runs n times.
- Inner loop runs up to (n − 1) times for each outer iteration.
- Total comparisons ≈ n × (n − 1) ≈ n^2.

So the brute-force closest pair algorithm has a time complexity of $O(n^2)$ — quadratic time.

c) Why Performance Degrades With Large Datasets

The quadratic growth rate directly explains the delays observed:

Non-linear scaling: Because complexity is $O(n^2)$, doubling the number of delivery points does not double the work — it quadruples it. For example:

- 100 points → ~4,950 comparisons
- 1,000 points → ~499,500 comparisons (≈100× more, not 10×)
- 10,000 points → ~49,995,000 comparisons

Redundant computation: The algorithm re-examines the entire dataset for every single point, ignoring spatial structure — e.g., two points on opposite sides of a city are still compared even though they are obviously not the closest pair.

No pruning or spatial partitioning: Brute-force has no way to eliminate far-apart points early. It lacks any divide-and-conquer or spatial indexing (such as grids, k-d trees, or sorting by x-coordinate) that could reduce comparisons.

Practical impact: As a real-world delivery company scales to thousands of drop points, $O(n^2)$ growth quickly overwhelms processing time, causing the delays observed. This is why real systems use the divide-and-conquer closest pair algorithm, which achieves O(n log n) by recursively splitting the point set, solving smaller subproblems, and merging results while only checking a narrow "strip" of points near the dividing line — avoiding unnecessary pairwise comparisons.

Summary Link: *The root cause of degraded performance is the algorithmic growth rate itself ($O(n^2)$), not just the hardware — meaning no amount of extra computing power fully solves the problem; a better algorithm is needed as data scales.*

d) Worked Numerical Example

To make the brute-force process concrete, consider a small delivery fleet with 5 GPS-tagged drop points (coordinates in km, relative to a depot at the origin):

Point	X (km)	Y (km)
P1	2	3
P2	5	4
P3	9	6
P4	4	7
P5	8	1

With n = 5 points, the brute-force algorithm evaluates C(5, 2) = 10 pairwise distances:

- $d(P1, P2) = \sqrt{[(5-2)^2 + (4-3)^2]} = \sqrt{10} \approx 3.16$ km
- $d(P1, P3) = \sqrt{[(9-2)^2 + (6-3)^2]} = \sqrt{58} \approx 7.62$ km
- $d(P1, P4) = \sqrt{[(4-2)^2 + (7-3)^2]} = \sqrt{20} \approx 4.47$ km
- $d(P1, P5) = \sqrt{[(8-2)^2 + (1-3)^2]} = \sqrt{40} \approx 6.32$ km
- $d(P2, P3) = \sqrt{[(9-5)^2 + (6-4)^2]} = \sqrt{20} \approx 4.47$ km
- $d(P2, P4) = \sqrt{[(4-5)^2 + (7-4)^2]} = \sqrt{10} \approx 3.16$ km
- $d(P2, P5) = \sqrt{[(8-5)^2 + (1-4)^2]} = \sqrt{18} \approx 4.24$ km
- $d(P3, P4) = \sqrt{[(4-9)^2 + (7-6)^2]} = \sqrt{26} \approx 5.10$ km
- $d(P3, P5) = \sqrt{[(8-9)^2 + (1-6)^2]} = \sqrt{26} \approx 5.10$ km
- $d(P4, P5) = \sqrt{[(8-4)^2 + (1-7)^2]} = \sqrt{52} \approx 7.21$ km

The smallest value is tied between d(P1, P2) and d(P2, P4), both ≈ 3.16 km — so {P1, P2} (or equivalently {P2, P4}) is reported as the closest pair. Note that even for this tiny fleet, 10 distance calculations were needed; this is exactly C(5, 2) = 5×4/2 = 10, confirming the $O(n^2)$ formula derived in part (b).

e) Pseudocode for the Brute-Force Algorithm

```
FUNCTION BruteForceClosestPair(points[1..n]):     minDist ← ∞     closestPair ← NULL
FOR i ← 1 TO n − 1:         FOR j ← i + 1 TO n:             dist ←
EuclideanDistance(points[i], points[j])             IF dist < minDist:
minDist ← dist                 closestPair ← (points[i], points[j])     RETURN closestPair,
minDist
```

Note the inner loop starts at j = i + 1 (not j = 1) to avoid comparing a point to itself and to avoid recomputing the same pair twice (e.g., (P1, P2) and (P2, P1) are the same pair). This halves the raw n × n comparisons to n(n − 1)/2, but the asymptotic complexity remains $O(n^2)$ since constant factors do not change the growth class.

f) Comparing Brute-Force vs. Divide-and-Conquer

Aspect	Brute-Force	Divide-and-Conquer
Time Complexity	$O(n^2)$	O(n log n)
n = 1,000	~499,500 comparisons	~9,966 operations
n = 100,000	~5×10^9 comparisons	~1.66×10^6 operations
Approach	Exhaustive pairwise check	Recursive split + strip merge

Scalability	Poor for large fleets	Suitable for large-scale routing

This comparison reinforces the conclusion from part (c): for a delivery company whose number of drop points grows into the thousands or more, migrating from the brute-force method to a divide-and-conquer (or spatial-indexing) approach is essential to keep closest-pair queries fast and responsive.